User File Descriptor table

- Each process has a user file descriptor table to identify all opened files.
- An entry in user file descriptor table pointer to an entry of kernel's global file table.
- Entry 0: standard input
- Entry 1: standard output
- Entry 2: error output

Unix I/O system calls

Unix I/O system calls

Most Unix I/O can be performed using the system calls:

- `open`: to open a file
- `creat`: to create a new file or rewrite an existing one
- `read`: to read a number of bytes
- `write`: to write a number of bytes
- `lseek`: to explicitly position the file offset
- `close`: to close a file

In contrast to the standard I/O functions, the Unix I/O system calls are unbuffered.

File descriptors

The kernel refers to any open file by a file descriptor, a nonnegative integers.

File descriptors range from 0 through `OPEN_MAX`, the total number of files a process can open.

The standard input (`stdin`), standard output (`stdout`) and standard error (`stdout`) have descriptors 0, 1 and 2 respectively.
The symbolic constants, defined in `<unistd.h>`, for these values are `STDIN_FILENO`, `STDOUT_FILENO` and `STDERR_FILENO`.

open() system call

open() synopsis

```
int open(const char *fName, int oflag[, int mode])
```

Returns file descriptor if OK, -1 otherwise.

The argument `oflag` is formed by bitwise **OR**'ing together 1 or more of the following constants (in < *fcntl.h* >)

- `O_RDONLY` : Open for reading only
- `O_WRONLY` : Open for writing only
- `O_RDWR` : Open for reading and writing only
- `O_APPEND` : Open for writing after the end of file
- `O_CREAT` : Create a file

Note that the third argument, only used when a file is created, supplies the initial file's permission flag settings, as an octal value.

Examples

- `if ((d=open(``data.txt'', O_RDONLY))==-1)`
  `fatalError();`
- `d=open(name,O_CREAT|O_WRONLY,0700)` In this case, 0700 means give all rights to the owner of this file and no permission to any other user.
  Example values for mode :
  - 0400: Allow read by owner
  - 0200: Allow write by owner
  - 0100: Allow execute
  - 0040: Allow read by group
  - 0004: Allow read by others
  - 0777: Allow read/write/execute by all

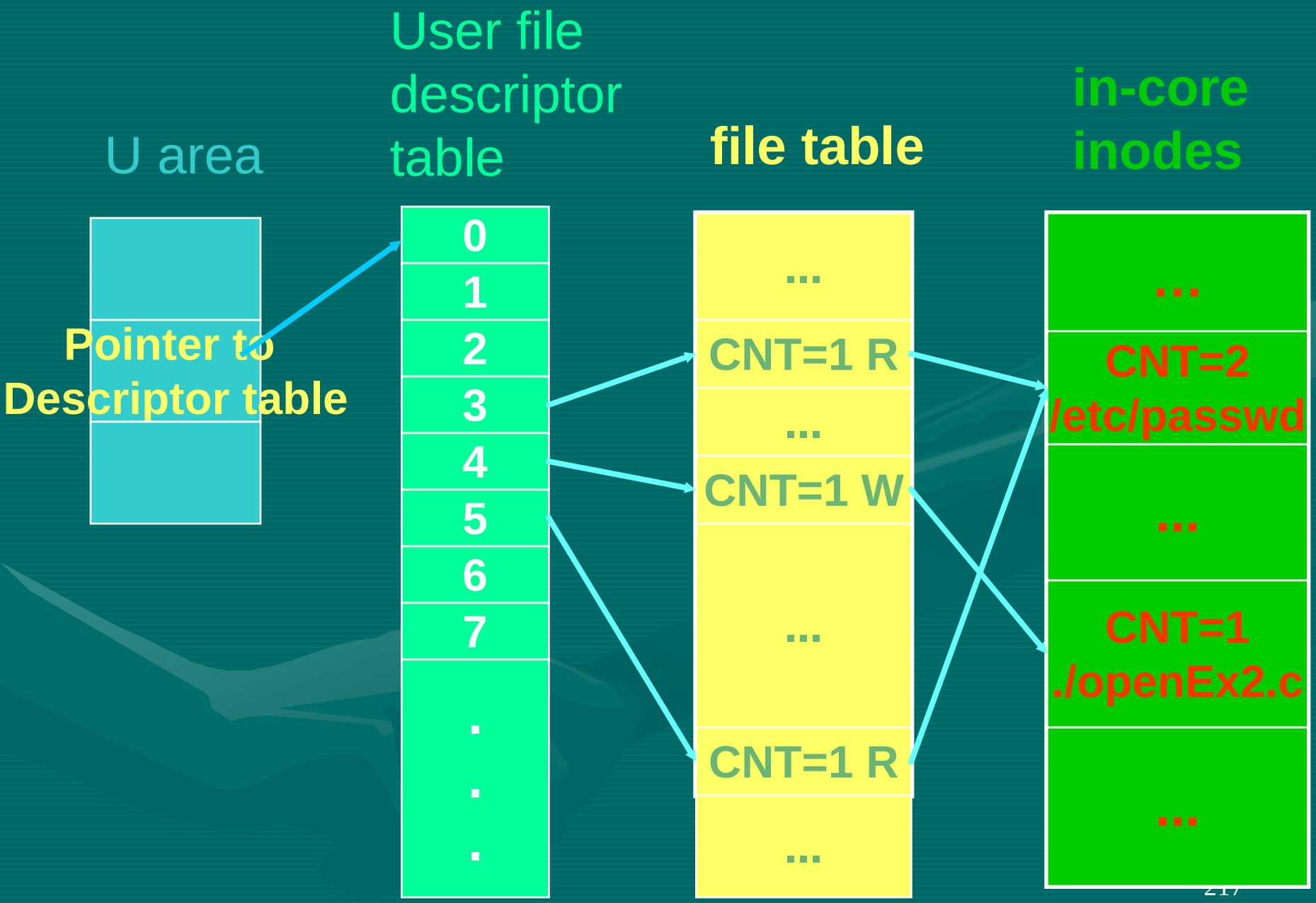
  Check the utility `umask` and `chmod`

Example: open

```
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>

main()
{
   int fd1, fd2, fd3;
   printf("Before open ...\n");
   fd1 = open("/etc/passwd", O_RDONLY);
   fd2 = open("./openEx1.c", O_WRONLY);
   fd3 = open("/etc/passwd", O_RDONLY);
   printf("fd1=%d  fd2=%d  fd3=%d \n", fd1, fd2, fd3);
}
```

```
$ cc openEx1.c -o openEx1
$ openEx1
Before open ...
fd1=3  fd2=4  fd3=5
$
```


U area
User file descriptor table
file table
in-core inodes
Pointer to Descriptor table
0
1
2
3
4
5
6
7
...
CNT=1 R
...
CNT=1 W
...
CNT=1 R
...
...
CNT=2 /etc/passwd
...
CNT=1 ./openEx2.c
...

`read()` and `write()` system calls

`read()` synopsis

```
ssize_t read(int fd, void *buf, size_t nbyte);
```

Reads as many bytes as it can, up to `nbyte`, and returns the number of bytes actually read.

`ssize_t` and `size_t` are usually defined as long integer(larger than an integer).

The value returned by `read()` can be:

- -1: in case of an error
- smaller than `nbyte`: the number of bytes left before the end of file was less than `nbyte` or, when reading from a keyboard where up to a line is normally read or, when reading from a network.
- 0: the end of file has been already reached

`write()` synopsis

```
ssize_t write(int fd, const void *buf, size_t
nbyte);
```

`write` returns `nbyte` if OK and -1 otherwise.

`int close(int fd)`

- `int close(int fd)` frees the file descriptor fd.
- If no other file descriptor is associated with a particular open file, the kernel frees all resources that were used for that file.
- `close()` returns 0 when OK and -1 otherwise.
  For example, -1 will be returned if `fd` was already closed.

System Call: close

- close: A process may close a file by close system call
- syntax:

  close(fd);

  A. fd: file descriptor

Example

System Call: read

- read: A process may read an opened file
- syntax:

  fd = read(fd, buffer, count);

  A. fd: file descriptor

  B. buffer: data to be stored in

  C. count: the number (count) of bytes
- Example

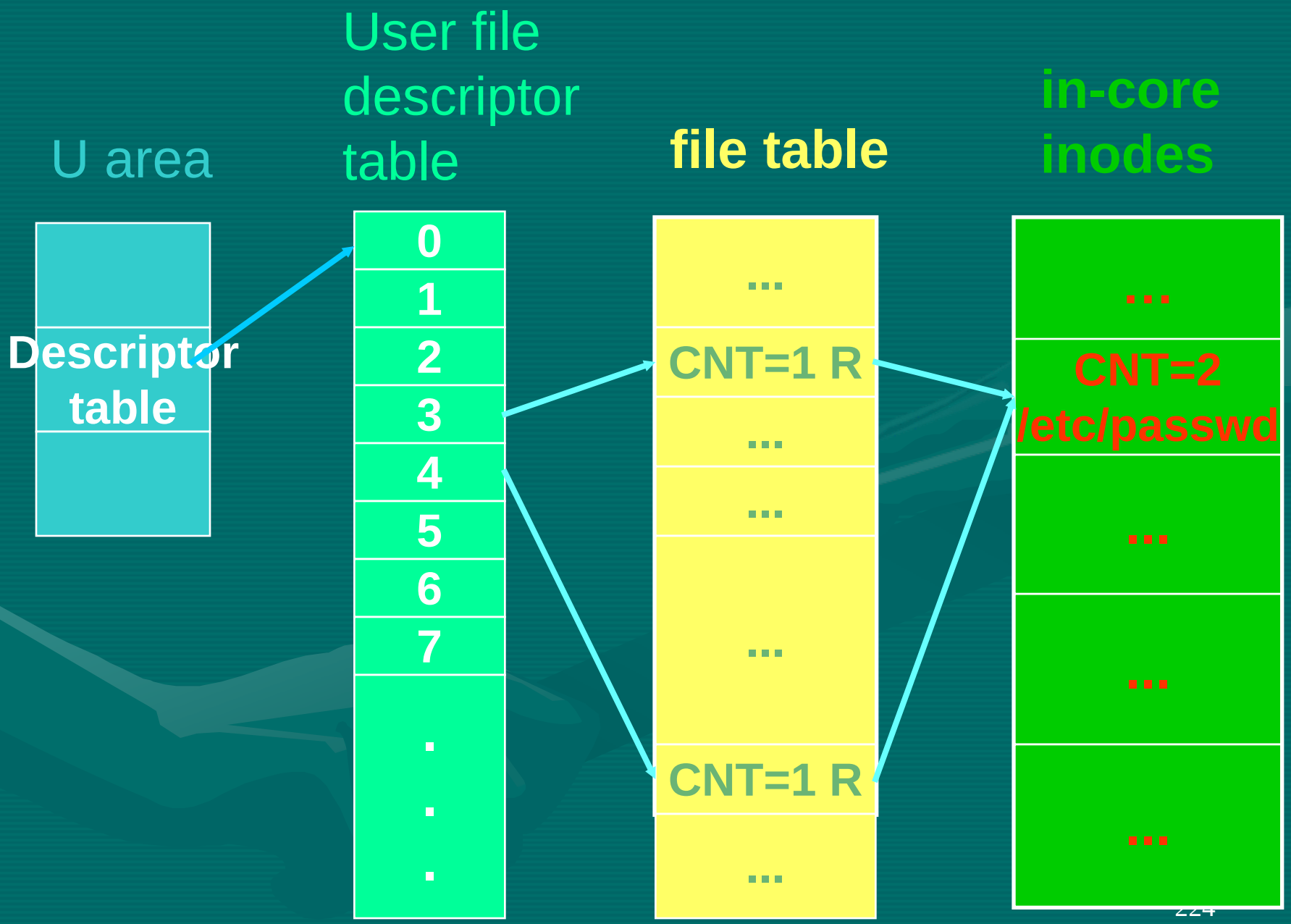
```
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>

main()
{
   int fd1, fd2, fd3;
   char buf1[20], buf2[20];
   buf1[19]='\0';
   buf2[19]='\0';
   printf("=======\n");
   fd1 = open("/etc/passwd", O_RDONLY);
   read(fd1, buf1, 19);
   printf("fd1=%d  buf1=%s \n",fd1, buf1);
   read(fd1, buf2, 19);
   printf("fd1=%d  buf2=%s \n",fd1, buf2);
   printf("=======\n");
}
```

```
$ cc openEx2.c -o openEx2
$ openEx2
=======
fd1=3  buf1=root:x:0:1:Super-Us
fd1=3  buf2=er:/:/sbin/sh
daemo
=======
$
```

```
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>
main()
{
   int fd1, fd2, fd3;
   char buf1[20], buf2[20];
   buf1[19]='\0';
   buf2[19]='\0';
   printf("======\n");
   fd1 = open("/etc/passwd", O_RDONLY);
   fd2 = open("/etc/passwd", O_RDONLY);
   read(fd1, buf1, 19);
   printf("fd1=%d  buf1=%s \n",fd1, buf1);
   read(fd2, buf2, 19);
   printf("fd2=%d  buf2=%s \n",fd2, buf2);
   printf("======\n");
}
```

```
$ cc openEx3.c -o openEx3
$ openEx3
======
fd1=3  buf1=root:x:0:1:Super-Us
fd2=4  buf2=root:x:0:1:Super-Us
======
$
```


U area
User file descriptor table
file table
in-core inodes
Descriptor table
0
1
2
3
4
5
6
7
...
CNT=1 R
...
...
...
CNT=1 R
...
...
CNT=2
/etc/passwd
...
...
...

Example of a copy utility

```
#include <unistd.h> #include <fcntl.h>

int main(int argc, char *argv[]){
 int fd1, fd2;
 char buffer[100];
 long int n1;

 if(((fd1 = open(argv[1], O_RDONLY)) == -1) ||
     ((fd2 = open(argv[2], O_CREAT|O_WRONLY|O_TRUNC,
                  0700)) == -1)){
    perror("file problem ");
    exit(1);
 }

 while((n1=read(fd1, buffer, 100)) > 0)
   if(write(fd2, buffer, n1) != n1){
      perror("writing problem ");
      exit(3);
   }


// Case of an error exit from the loop

 if(n1 == -1){
   perror("Reading problem ");
   exit(2);
 }
 close(fd2);
 exit(0);
}
```

lseek() system call

lseek() synopsis

```
off_t lseek(int fd, off_t offset, int whence);
```

Return the resulting offset if OK, -1 otherwise.
The return type `off_t` is a long integer.

Similar to `fseek()`, it sets the file pointer(position) associated with the open file descriptor specified by the file descriptor `fd` as follows:

- If whence is SEEK_SET, the pointer is set to offset bytes.
- If whence is SEEK_CUR, the pointer is set to its current location plus offset.
- If whence is SEEK_END, the pointer is set to the size of the file plus offset.

These three constant are defined in < *unistd.h* >.

Note

`lseek()` allows the file pointer to be set beyond the existing data in the file. If data are later written at this point, subsequent reads in the gap between the previous end of data and the newly written data will return bytes of value 0 until data are written into the gap.

Example

Reversing a file

Write a C program, called reverse, to take a file as input then copies it to another file in reverse order. That is, the last byte becomes the first, the byte just just before the last one becomes the second, etc.

The program call should look like:

```
reverse fileIn fileOut
```

There are two solutions,

- either we use `lseek()` on `fileIn` (read bytes from the end), or
- we use `lseek()` on `fileOut`. (write bytes starting at the end)

```
#include <unistd.h> #include <fcntl.h>

// solution 1: Start reading  at the end
//             Use lseek() on fileIn
int main(int argc, char *argv[]){
 int fd1, fd2;
 char buffer; // 1 character buffer
 long int i=0, fileSize=0;

 fd1=open(argv[1], O_RDONLY);
 fd2=open(argv[2], O_CREAT|O_WRONLY|O_TRUNC, 0755);

 while(read(fd1, &buffer, 1)>0)
   fileSize++;
 while(++i <= fileSize){
   lseek(fd1, -i, SEEK_END);
   read(fd1, &buffer, 1);
   write(fd2, &buffer, 1);
 }
 close(fd1);
 close(fd2);
}
```

```
//solution 2: start writing at end
int main(int argc, char *argv[]){
 int fd1, fd2;
 char buffer; // 1 character buffer
 long int i=0, fileSize=0;

 fd1=open(argv[1], O_RDONLY);
 fd2=open(argv[2], O_CREAT|O_WRONLY|O_TRUNC, 0755);

 while(read(fd1, &buffer, 1)>0)
   fileSize++;
 lseek(fd2, fileSize-1, SEEK_SET);//pointer to end
 lseek(fd1, 0, SEEK_SET);   // rewind fd1!!
 while(++i <= fileSize){
   read(fd1, &buffer, 1);
   lseek(fd2, -i, SEEK_END);
   write(fd2, &buffer, 1);
 }
 close(fd1);
 close(fd2);
}
```

System Call: dup

- dup: copy a file descriptor into the first free slot of the user file descriptor table.
- syntax:

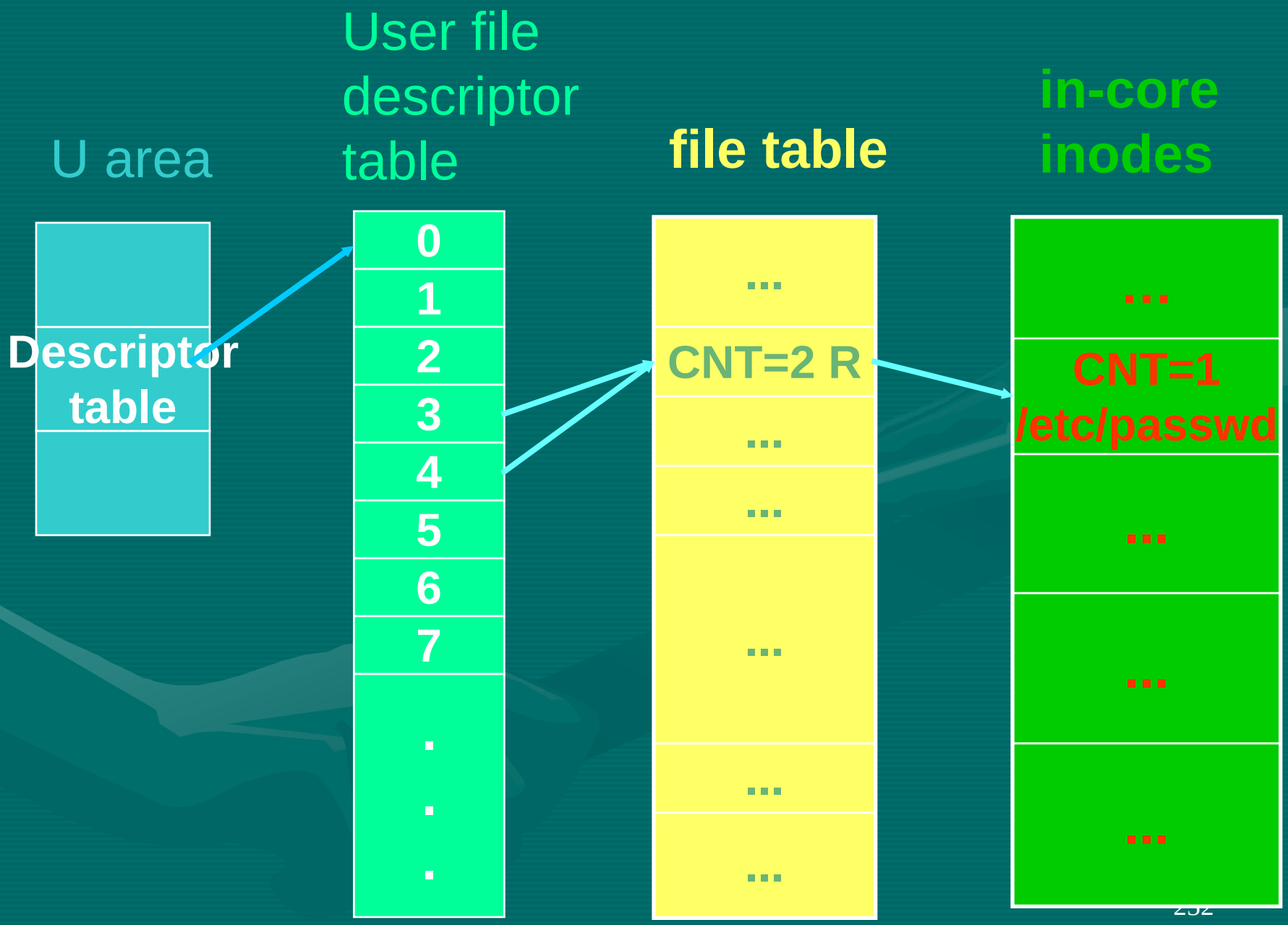
  newfd = dup(fd);

  A. fd: file descriptor

Example

```
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>
main()
{
   int fd1, fd2, fd3;
   char buf1[20], buf2[20];
   buf1[19]='\0';
   buf2[19]='\0';
   printf("======\n");
   fd1 = open("/etc/passwd", O_RDONLY);
   fd2 = dup(fd1);
   read(fd1, buf1, 19);
   printf("fd1=%d  buf1=%s \n",fd1, buf1);
   read(fd2, buf2, 19);
   printf("fd2=%d  buf2=%s \n",fd2, buf2);
   printf("======\n");   char buf1[20], buf2[20];
}
```

```
$ cc openEx4.c -o openEx4
$ openEx4
======
fd1=3  buf1=root:x:0:1:Super-Us
fd2=4  buf2=er:/:/sbin/sh
daemo
======
$
```


U area
User file descriptor table
file table
in-core inodes
Descriptor table
0
1
2
3
4
5
6
7
CNT=2 R
CNT=1
/etc/passwd

System Call: creat

- creat: A process may create a new file by creat system call
- syntax:

  fd = write(pathname, mode);

  A. pathname: file name

  B. mode: read/write

  Example

System Call: write

- write: A process may write data to an opened file
- syntax:

  fd = write(fd, buffer, count);

  A. fd: file descriptor

  B. buffer: data to be stored in

  C. count: the number (count) of byte
- Example

```
/* creatEx1.c */
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>
main()
{
   int fd1;
   char *buf1="I am a string\n";
   char *buf2="second line\n";
   printf("======\n");
   fd1 = creat("./testCreat.txt", O_WRONLY);
   write(fd1, buf1, 20);
   write(fd1, buf2, 30);
   printf("fd1=%d  buf1=%s \n",fd1, buf1);
   close(fd1);
   chmod("./testCreat.txt", 0666);
   printf("======\n");
}
```

```
$ cc creatEx1.c -o creatEx1
$ creatEx1
======
fd1=3  buf1=I am a string

======
$ ls -l testCreat.txt
-rw-rw-rw-   1 cheng    staff         50 May 10 20:37 testCreat.txt
$ more testCreat.txt
...
```

System Call: stat/fstat

- stat/fstat: A process may query the status of a file (locked) file type, file owner, access permission. file size, number of links, inode number, access time.
- syntax:

  stat(pathname, statbuffer); fstat(fd, statbuffer);

  A. pathname: file name

  B. statbuffer: read in data

  C. fd: file descriptor

Example

```
/* statEx1.c */
#include <sys/stat.h>
main()
{
   int fd1, fd2, fd3;
   struct stat bufStat1, bufStat2;
   char buf1[20], buf2[20];
   printf("======\n");
   fd1 = open("/etc/passwd", O_RDONLY);
   fd2 = open("./statEx1", O_RDONLY);
   fstat(fd1, &bufStat1);    fstat(fd2, &bufStat2);
   printf("fd1=%d inode no=%d block size=%d blocks=%d\n",
   fd1, bufStat1.st_ino,bufStat1.st_blksize, bufStat1.st_blocks);
   printf("fd2=%d inode no=%d block size=%d blocks=%d\n",
   fd2, bufStat2.st_ino,bufStat2.st_blksize, bufStat2.st_blocks);
   printf("======\n");
}
```

```
$ cc statEx1.c -o statEx1
$ statEx1
======
fd1=3 inode no=21954 block size=8192 blocks=6
fd2=4 inode no=190611 block size=8192 blocks=
======
...
```

System Call: link/unlink

- link: hardlink a file to another

- syntax:
  link(sourceFile, targetFile); unlink(file)
  A. sourceFile targetFile, file: file name
  Example:
  Lab exercise: write a c program which use link/unlink system call. Use ls -l to see the reference count.

Inheriting and changing directory

A child process inherits its current working directory from its parent.
Each process can change its working directory using `chdir()`.

`chdir()` synopsis:

```
int chdir(const char * pathName);
```

`chdir()` returns 0 if successful -1 otherwise.
It fails if the specified path name does not exist or if the process does not have execute permission from the directory.

System Call: chdir

- chdir: A process may change the current directory of a processl
- syntax:

  chdir(pathname);

  A. pathname: file name

  Example

```
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>

main()
{
   chdir("/usr/bin");
   system("ls -l");
}
```

```
$ ls -l /usr/bin
$
```

System scheduling priority

- Every process has a system scheduling priority.
- Each process runs at a default system priority: the priority inherited from its parent.
- The numerical range of priority values is: -20 to 19 (Only the super-user can get negative values). However, this range differs from one Unix platform to another.
- The scheduling priority can be changed using the `nice()` system call.
- Priority value can also be changed using `setpriority()`.
- Priority value can be obtained using `getpriority()`.

The priority value affects the amount of CPU time allocated to the process. The smaller the value, the faster the process will be.

Changing priority `nice()`

Synopsis:

```
int nice(int delta);
```

`nice()` adds `delta` to the process current priority value. Note that only super-user processes can have a negative priority value.

`nice()` returns the new priority value if successful and -1 otherwise.

getpriority() and setpriority()

getpriority()

Synopsis: `int getpriority(int which, id_t who);`
obtains the current scheduling priority of a process, process group, or user. Returns the priority value on success, -1 on failure.

setpriority()

Synopsis: `int setpriority(int which, id_t who, int priority);`
sets the scheduling priority of a process, process group, or user. Returns 0 on success, -1 on failure.

- The two functions require `#include <sys/resource.h>`
- `which` identifies whether it is a process (`PRIO_PROCESS`), process group...etc.
- `who` interpreted as a process id, group id,...etc. If `who` is 0, the calling process, (or group,...etc) is considered.
- `priority` is the priority value (in the appropriate range).

Example...

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
  int pid;
  printf("I am the parent %d. My priority is %d\n",
          getpid(),getpriority(PRIO_PROCESS,getpid()));

  if ((pid = fork())==0){
      printf("I am the child %d. My priority is %d\n",
            getpid(),getpriority(PRIO_PROCESS,getpid()));
      printf("Child: nice(19) returns %d\n",nice(19));
      printf("I am the child %d. My priority is now %d\n",
            getpid(),getpriority(PRIO_PROCESS,getpid()));
      exit(20);
  }
  else{
      sleep(5);
      printf("Changing parent's priority...\n");
      printf("Parent: nice(10) returns %d\n",nice(10));
      int status;
      int term_pid = wait(&status);

      int new_pid = fork();
      if (new_pid == 0){
        printf("I am the new child %d. My priority is %d\n",
                getpid(),getpriority(PRIO_PROCESS,getpid()));
        printf("New child: nice(5) returns %d\n",nice(5));
        printf("I am the new child %d. My priority is now %d\n",
                getpid(),getpriority(PRIO_PROCESS,getpid()));
        exit(21);

      }
      else term_pid = wait(&status);
  }
}
```

The output of the previous program is:

```
I am the parent 9306. My priority is 0

I am the child 9307. My priority is 0

Child: nice(19) returns 19

I am the child 9307. My priority is now 19

Changing parent's priority...

Parent: nice(10) returns 10

I am the new child 9308. My priority is 10

New child: nice(5) returns 15

I am the new child 9308. My priority is now 15
```

Example...

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
  int pid;
  printf("I am the parent %d. My priority is %d\n",
            getpid(),getpriority(PRIO_PROCESS,getpid()));

  if ((pid = fork())==0){
      printf("I am the child %d. My priority is %d\n",
            getpid(),getpriority(PRIO_PROCESS,getpid()));
      setpriority(PRIO_PROCESS,getpid(),19);
      printf("I am the child %d. My priority is now %d\n",
            getpid(),getpriority(PRIO_PROCESS,getpid()));
      exit(20);
  }
  else{
      sleep(5);
      printf("Changing parent's priority...\n");
      printf("Parent: nice(10) returns %d\n",nice(10));
      int status;
      int term_pid = wait(&status);
  }
}
```

The output of the previous program is:

```
I am the parent 13557. My priority is 0

I am the child 13558. My priority is 0

I am the child 13558. My priority is now 19

Changing parent's priority...

Parent: nice(10) returns 10
```

Process groups

- Every process is a member of a process group.
- A child inherits its process group from its parent.
- When a process execs, its process group remains the same.
- A process may change its process group to a new value: either to become a member of another group or to create its own group (in which the process will initially be the leader and sole member).
- One of the group members is the group leader.
- Each member of the group has the group leader's process-ID as its process-group-ID.

- The kernel provides a system call to send a signal to each member of a designated process group.
- This would be used to terminate the entire group as a whole, but any signal can be broadcast in this way.

setpgid()

```
#include <sys/types.h>
#include <unistd.h>
```

setpgid()

Synopsis: `int setpgid(pid_t pid,pid_t pgid)`

- sets the process group ID of the process, whose ID is `pid`, to `pgid`. It returns 0 if successful and -1 otherwise.
- If `pgid` is equal to `pid`, the process becomes a process group leader.
- If `pid` is equal to 0, the calling process group ID is set to `pgid`.
- if `pgid` is equal 0, the process specified by `pid` becomes a process group leader.

getpgid()

```
#include <sys/types.h>
#include <unistd.h>
```

getpgid()

Synopsis: `pid_t getpgid(pid_t pid)`
returns the process group ID of the process with PID equal to `pid`. If `pid` is 0, the calling process group ID is returned.

Example

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/wait.h>

int main(int argc, char *argv[]){

 int pid = getpid();
 printf("Parent: PID = %d, PPID = %d, PGID = %d\n",pid,getppid(),getpgid(pid));

 if ((pid = fork())==0){

    printf("Child: PID = %d, PPID = %d, PGID = %d\n",
            getpid(),getppid(),getpgid(getpid()));
    exit(1);
 }
 sleep(5);
}
```

Output:

```
Parent: PID = 20814, PPID = 20381, PGID = 20814

Child: PID = 20815, PPID = 20814, PGID = 20814
```

Another example

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#include <sys/wait.h>

int main(int argc, char *argv[]){

 int pid = getpid();
 printf("Parent: PID = %d, PPID = %d, PGID = %d\n",pid,getppid(),getpgid(pid));

 if ((pid = fork())==0){

    printf("Child: PID = %d, PPID = %d, PGID = %d\n",getpid(),getppid(),getpgid(getpid()));

    setpgid(0,0); //or setpgid(getpid(),0);
    printf("Child after setpgid: PID = %d, PPID = %d, PGID = %d\n",
            getpid(),getppid(),getpgid(getpid()));

 }
 sleep(5);
}
```

Output:

```
Parent: PID = 22295, PPID = 20381, PGID = 22295

Child: PID = 22296, PPID = 22295, PGID = 22295

Child after setpgid: PID = 22296, PPID = 22295, PGID = 22296
```





End of Lecture on Linux System Programming

Compiling a C Program

- GNU's C Compiler (gcc)
  - The main compiler that will be used in this course
  - For compiling C++ programs, you can use g++
  - Linux/UNIX does not have a pretty program like Microsoft Visual Studio for managing C programming projects

- Compiling a C program

  gcc –g –m32 –ansi –Wall –c main.c –o main.o

  gcc –g –m32 –ansi –Wall –c numprint.c –o numprint.o

  -g: include symbols for gdb debugger.

  -m32: compile and link 32bit i386 binaries

  -ansi: check code against ansi C standard.

  -Wall: display all warnings.

  - Use –ansi and -Wall for user programs.
  - Use –Wall for kernel programs.

- Linking

  gcc –g –m32 –ansi –Wall main.o numprint.o –o lab01

Makefile

- You need to type quite a bit for compiling a simple program
- For large projects
  - You may have many .c and .h files
  - You may use many library calls
  - You need to specify them at compilation time.
  - Things start to get tedious and messy
- One way to manage this complexity is to use a Makefile
  - Automates the compilation process
  - Easy to declare all the compilation options and flags

Makefile

- Example Makefile

```
SRCS = main.c numprint.c
HDRS = main.h
OBJECTS = main.o numprint.o
INCLUDE = -I/usr/local/include
LIBS = -lm
CC = gcc
CFLAGS = -g –ansi –Wall
EXEC = lab01
all:   $(EXEC)
$(EXEC):      $(OBJECTS)
   $(CC) $(FLAGS) $(INCLUDE) $(LIBS) $(OBJECTS) –o $(EXEC)
clean:
   rm –f $(OBJECTS) $(EXEC) core *~
depend:
   makedepend -- $(CFLAGS) $(INCLUDES) $(SRCS) $(HDRS) –

# DO NOT DELETE THIS LINE – make depend depends on it.
```

Makefile

- The first few lines are fairly straightforward
  - SRCS, HDRS, and OBJECTS specify the source, header, and object files
  - INCLUDE, the directory for include files
  - LIBS, the library to be linked into the compilation
  - CC, the type of C compiler
  - CFLAGS, the compilation flags
  - EXEC, the name of the executable image.
- The line containing *all* specifies the final compilation targets
  - in this case, the content of EXEC, or $(EXEC).
- The creation of $(EXEC) depends on the $(OBJECTS), or object files.
- To create $(EXEC), the compiler needs to link the objects by running the $(CC) command.
- All the .c files are automatically converted to .o files without the need of specifications.
- Note that you need to **tab** the indentation of the $(CC) command. Makefile won't work if you use the space bar to create the indentation.
- Lines start with # denote comments. Let's ignore the remaining lines for now. With this Makefile in your current project directory, all you need to type to compile your project is ***gmake***.

Makefile

- Prepare a Makefile (can be found on course web)

  cp /course/comp252/web08fall/Lab1/Makefile/Makefile .

  cp /course/comp252/web08fall/Lab1/Makefile/numprint.c .

  cp /course/comp252/web08fall/Lab1/Makefile/main.h .

  cp /course/comp252/web08fall/Lab1/Makefile/main.c .

  cat Makefile
- Run the Makefile
  - gmake
- More about Makefile
  - http://users.actcom.co.il/~choo/lupg/tutorials/writing-makefiles/writing-makefiles.html
  - http://www.gnu.org/software/make/manual/html_node/index.html